
RÉPUBLIQUE ALGÉRIENNE DÉMOCRATIQUE ET
POPULAIRE
MINISTÈRE DE L'ENSEIGNEMENT SUPÉRIEUR ET DE LA
RECHERCHE SCIENTIFIQUE



UNIVERSITÉ D'ALGER 1 - BENYOUCEF BENKHEDDA

Rapport de Mini-Projet
Apprentissage Fédéré pour la Reconnaissance
des Caractères Arabes Manuscrits
Deep Learning - Classification Hors Ligne

Module : Machine Learning Avancé

Réalisé par :

ZERROUKI Imane	Groupe 02
YAHIAOUI Mohamed Islem	Groupe 02
SIKENE Raihane	Groupe 02
MELLALI Abderhmane Rayane	Groupe 02
BOUGOUFFA Youcef	Groupe 01

Enseignants :

Dr. BOUFENAR Chaouki
Mme. BOUKHIAR Naima

2^{ème} Année Master – Analyse et Sciences de Données

Année Universitaire 2025/2026

Table des matières

1	Introduction	3
1.1	Contexte et Motivation	3
1.2	Objectifs du Projet	3
1.3	Structure du Projet	3
2	Apprentissage Fédéré : Fondements Théoriques	3
2.1	Définition et Principes	3
2.2	Algorithme FedAvg	3
3	Reconnaissance de Caracteres Arabes	4
4	Scenarios IID vs Non-IID	4
4.1	Données IID (Independent and Identically Distributed)	4
4.2	Données Non-IID	4
5	Ensembles de Données	4
5.1	AHCD (Arabic Handwritten Characters Dataset)	4
5.2	OIHACDB (Off-line Isolated Handwritten Arabic Characters Database)	5
6	Preprocessing	5
7	Architecture du Systeme	5
8	Modele CNN	5
9	Hyperparametres	6
10	Apprentissage Centralisé	6
11	Apprentissage Fédéré - FedAvg	7
12	Resultats Experimentaux	8
13	Mise en Place Experimentale	8
14	Resultats du Modele Centralisé	8
15	Resultats FL - Scenario IID	9
15.1	AHCD	9
15.2	OIHACDB	9

16 Resultats FL - Scenario Non-IID	9
16.1 AHCD	9
16.2 OIHACDB	10
17 Analyse Comparative	10
17.1 Observations Cles	10
18 Visualisation des Resultats	11
18.1 Convergence FL IID - AHCD	11
18.2 Convergence FL Non-IID - AHCD	11
18.3 Comparaison IID vs Non-IID - AHCD	12
18.4 Comparaison Finale - AHCD	13
18.5 Comparaison OIHACDB	14
18.6 Master Comparaison	14
19 Resume des Resultats	15
19.1 Resultats Cles	15
19.2 Efficacite de FedAvg	15
20 Contributions	15
21 Limitations	15
22 Travaux Futurs	16
23 Remarques Finales	16

1 Introduction

1.1 Contexte et Motivation

La reconnaissance de caractères manuscrits arabes représente un domaine de recherche crucial dans le traitement automatique des images et des documents. L'entraînement des modèles de machine learning nécessite généralement de centraliser d'importantes quantités de données, ce qui pose des problèmes de confidentialité et de sécurité, particulièrement dans les contextes gouvernementaux et financiers.

L'Apprentissage Fédéré (FL) est une solution permettant d'entraîner des modèles sans centraliser les données. Ce travail explore l'application du FL pour la reconnaissance de caractères arabes manuscrits isolés.

1.2 Objectifs du Projet

- Implémenter un système complet d'apprentissage fédéré pour la reconnaissance de caractères arabes
- Comparer les performances de l'apprentissage centralisé et fédéré
- Évaluer l'impact de la distribution des données (IID vs Non-IID) sur la précision
- Analyser l'efficacité de communication et la convergence dans un cadre fédéré

1.3 Structure du Projet

Le projet est organisé en quatre modules principaux :

1. Preprocessing des donnees : Normalisation des donnees AHCD et OIHACDB
2. Modeles : Implementation de CNN avec TensorFlow/Keras
3. Distribution federee : Division des donnees sur 10 clients avec scenarios IID et Non-IID
4. Entrainement : Implementation de l'apprentissage centralise et de l'algorithme FedAvg

2 Apprentissage Fédéré : Fondements Théoriques

2.1 Définition et Principes

L'Apprentissage Federe est un paradigme d'apprentissage distribue ou les donnees ne quittent jamais les dispositifs clients. Le serveur central coordonne l'entrainement en :

1. Distribuant le modele global aux clients
2. Permettant a chaque client d'entraîner localement sur ses donnees privees
3. Agregant les mises a jour de poids des clients au serveur
4. Mettant a jour le modele global avec la moyenne des mises a jour

2.2 Algorithme FedAvg

L'algorithme FedAvg (Federated Averaging) est le plus largement utilise. A chaque round t :

$$w_t^{(k)} = w_{t-1} - \eta \nabla f_k(w_{t-1}) \quad (1)$$

ou $w_t^{(k)}$ est le poids mise a jour du client k , η est le taux d'apprentissage, et ∇f_k est le gradient local.

La mise a jour globale est :

$$w_t = \sum_{k=1}^K \frac{n_k}{N} w_t^{(k)} \quad (2)$$

ou n_k est la taille des donnees du client k et N est la taille totale des donnees.

3 Reconnaissance de Caracteres Arabes

Le script arabe presente plusieurs defis uniques :

- 28 caracteres de base sans distinction majuscules/minuscules
- Chaque caractere peut prendre 2 a 4 formes selon sa position (initiale, mediane, finale, isolee)
- Caracteres similaires distingues uniquement par des points diacritiques
- Variabilite significative dans l'ecriture manuscrite

4 Scenarios IID vs Non-IID

4.1 Données IID (Independent and Identically Distributed)

Les donnees IID distribuent uniformement les caracteres sur tous les clients. Cela represente un scenario ideal mais peu realiste ou tous les clients ont acces a un echantillon representative complet des donnees.

4.2 Données Non-IID

Les donnees Non-IID distribuent les caracteres de maniere heterogene, simulant la realite ou differents clients ont des ensembles de caracteres differents. Cela presente un defi plus grand pour la convergence de l'algorithme federe.

5 Ensembles de Données

5.1 AHCD (Arabic Handwritten Characters Dataset)

- Nombre d'images : 16,800
- Dimensions : 32 x 32 pixels
- Format : nuances de gris
- Classes : 28 caractères arabes
- Distribution : 600 échantillons par caractère

5.2 OIHACDB (Off-line Isolated Handwritten Arabic Characters Database)

- Nombre d'images : 5,600
- Dimensions : 128 x 128 pixels (redimensionnées en 32 x 32)
- Format : nuances de gris
- Classes : caractères arabes isolés

6 Preprocessing

Le processus de preprocessing comprend :

1. Chargement des images depuis les fichiers sources
2. Redimensionnement a 32 x 32 pixels pour uniformite
3. Conversion en nuances de gris
4. Normalisation en float32 avec valeurs entre 0 et 1 (division par 255)
5. Stockage en format .npz pour acces rapide

```

1 import numpy as np
2 import cv2
3
4 def preprocess_ahcd(data_path):
5     images = []
6     labels = []
7
8     for label, char_dir in enumerate(sorted(Path(data_path).iterdir())
9         ↪ ):
10         for img_path in char_dir.glob('*.bmp'):
11             img = cv2.imread(str(img_path), cv2.IMREAD_GRAYSCALE)
12             img = cv2.resize(img, (32, 32))
13             img = img.astype(np.float32) / 255.0
14             images.append(img)
15             labels.append(label)
16
17     return np.array(images), np.array(labels)

```

7 Architecture du Systeme

Notre systeme implemente une resultature client-serveur typique pour l'Apprentissage Federe :

1. Serveur Central : Maintient le modele global et coordonne les rounds d'entrainement
2. 10 Clients Fédérés : Chacun entraine localement sur ses donnees privees
3. Synchronisation : Communication a la fin de chaque round local pour agrégation

8 Modele CNN

mise en placature CNN pour la classification d'images :

- Couche d'entree : 32 x 32 images en nuances de gris (1 canal)
- Conv Layer 1 : 32 filtres de 3x3, activation ReLU
- MaxPooling 1 : Pool de 2x2
- Conv Layer 2 : 64 filtres de 3x3, activation ReLU
- MaxPooling 2 : Pool de 2x2
- Flatten : Aplatissement des features
- Dense Layer : 128 neurones, activation ReLU, Dropout 0.5
- Couche de Sortie : 28 neurones (softmax)

```

1 from tensorflow import keras
2 from tensorflow.keras import layers
3
4 def create_cnn_model(input_shape=(32, 32, 1), num_classes=28):
5     model = keras.Sequential([
6         layers.Input(shape=input_shape),
7         layers.Conv2D(32, (3, 3), activation='relu'),
8         layers.MaxPooling2D((2, 2)),
9         layers.Conv2D(64, (3, 3), activation='relu'),
10        layers.MaxPooling2D((2, 2)),
11        layers.Flatten(),
12        layers.Dense(128, activation='relu'),
13        layers.Dropout(0.5),
14        layers.Dense(num_classes, activation='softmax')
15    ])
16
17    model.compile(
18        optimizer='adam',
19        loss='sparse_categorical_crossentropy',
20        metrics=['accuracy']
21    )
22    return model

```

9 Hyperparametres

TABLE 1 – Hyperparametres d'entrainement

Parametre	Centralise	Federe
Nombre d'epoques	20	20 rounds
Epoques locales	-	5
Taille du batch	32	32
Taux d'apprentissage	0.001	0.001
Optimiseur	Adam	Adam
Fonction de perte	Sparse Categorical Crossentropy	Sparse Categorical Crossentropy
Nombre de clients	1	10

10 Apprentissage Centralisé

L'approche centralisee entraine un seul modele sur toutes les donnees combinees.

```

1 def train_centralized(X_train, y_train, X_test, y_test, epochs=20):
2     model = create_cnn_model()
3
4     history = model.fit(
5         X_train, y_train,
6         epochs=epochs,
7         batch_size=32,
8         validation_data=(X_test, y_test),
9         verbose=1
10    )
11
12    test_loss, test_acc = model.evaluate(X_test, y_test)
13    return model, history

```

11 Apprentissage Fédéré - FedAvg

L'algorithme FedAvg distribue l'entraînement sur 10 clients. A chaque round t :

1. Le serveur selectionne tous les clients
2. Le serveur distribue le modele global w_t a tous les clients
3. Chaque client entraine localement pendant 5 epoques
4. Chaque client envoie w_k au serveur
5. Le serveur agrege les poids : $w_{t+1} = \sum_{k=1}^K \frac{n_k}{N} w_k$

```

1 def federated_training(client_data, num_rounds=20):
2     global_model = create_cnn_model()
3     initial_weights = global_model.get_weights()
4
5     for round_num in range(num_rounds):
6         print(f"Round {round_num+1}/{num_rounds}")
7
8         client_weights = []
9         client_sizes = []
10
11        # Training local sur chaque client
12        for X_client, y_client in client_data:
13            local_model = create_cnn_model()
14            local_model.set_weights(initial_weights)
15
16            local_model.fit(X_client, y_client, epochs=5,
17                           batch_size=32, verbose=0)
18
19            client_weights.append(local_model.get_weights())
20            client_sizes.append(len(X_client))
21
22        # Agregation des poids (FedAvg)
23        total_size = sum(client_sizes)
24        aggregated_weights = []
25
26        for weight_idx in range(len(initial_weights)):

```



```

27     weighted_sum = None
28
29     for client_idx, client_weight in enumerate(client_weights)
30         ↪ :
31         weight = client_weight[weight_idx]
32         factor = client_sizes[client_idx] / total_size
33
34         if weighted_sum is None:
35             weighted_sum = factor * weight
36         else:
37             weighted_sum += factor * weight
38
39         aggregated_weights.append(weighted_sum)
40
41     global_model.set_weights(aggregated_weights)
42     initial_weights = aggregated_weights
43
44     return global_model

```

12 Resultats Experimentaux

13 Mise en Place Experimentale

- **Langage** : Python 3.8+
- **Framework** : TensorFlow 2.x, Keras
- **Dataset** : AHCD (16,800 images)
- **Split Train/Test** : 80/20
- **Nombre de Clients** : 10
- **Scenarios** : IID et Non-IID

14 Resultats du Modele Centralisé

TABLE 2 – Resultats de l’Apprentissage Centralise

Metrique	Valeur
Precision de Test (AHCD)	90.06%
Precision de Test (OIHACDB)	96%
Nombre d’epoques	10
Batch Size	64

15 Resultats FL - Scenario IID

15.1 AHCD

TABLE 3 – Resultats FL IID AHCD (20 Rounds)

Round	Train Accuracy	Test Accuracy
1	60%	55%
5	85%	82%
10	90%	88%
20	91.82%	91.82%

Precision Finale IID AHCD : 91.82% (+1.76% par rapport au centralise)

15.2 OIHACDB

TABLE 4 – Resultats FL IID OIHACDB (20 Rounds)

Round	Precision
1	65%
10	94%
20	98%

Precision Finale IID OIHACDB : 98% (+2% par rapport au centralise)

16 Resultats FL - Scenario Non-IID

16.1 AHCD

TABLE 5 – Resultats FL Non-IID AHCD (20 Rounds)

Round	Train Accuracy	Test Accuracy
1	50%	45%
5	68%	62%
10	72%	70%
20	72.68%	72.68%

Precision Finale Non-IID AHCD : 72.68% (degradation de 19.14%)

16.2 OIHACDB

TABLE 6 – Resultats FL Non-IID OIHACDB (20 Rounds)

Round	Precision
1	40%
10	62%
20	62%

Precision Finale Non-IID OIHACDB : 62% (degradation de 36%)

17 Analyse Comparative

TABLE 7 – Comparaison Complete : Centralise vs FL IID vs FL Non-IID

Dataset	Centralise	FL IID	FL Non-IID	Degradation
AHCD	90.06%	91.82%	72.68%	19.14%
OIHACDB	96%	98%	62%	36%

17.1 Observations Cles

FL IID Surpasse Centralise :

- AHCD : +1.76%
- OIHACDB : +2%
- La regularisation implicite via agregation distribuee ameliore la generalisation

FL Non-IID Souffre Severement :

- AHCD : Degradation de 19.14%
- OIHACDB : Degradation extreme de 36%
- L'heterogeneite des donnees est le goulot d'etranglement majeur

Impact Relatif :

- AHCD : 20.84% (19.14% / 91.82%)
- OIHACDB : 36.73% (36% / 98%)

18 Visualisation des Resultats

18.1 Convergence FL IID - AHCD

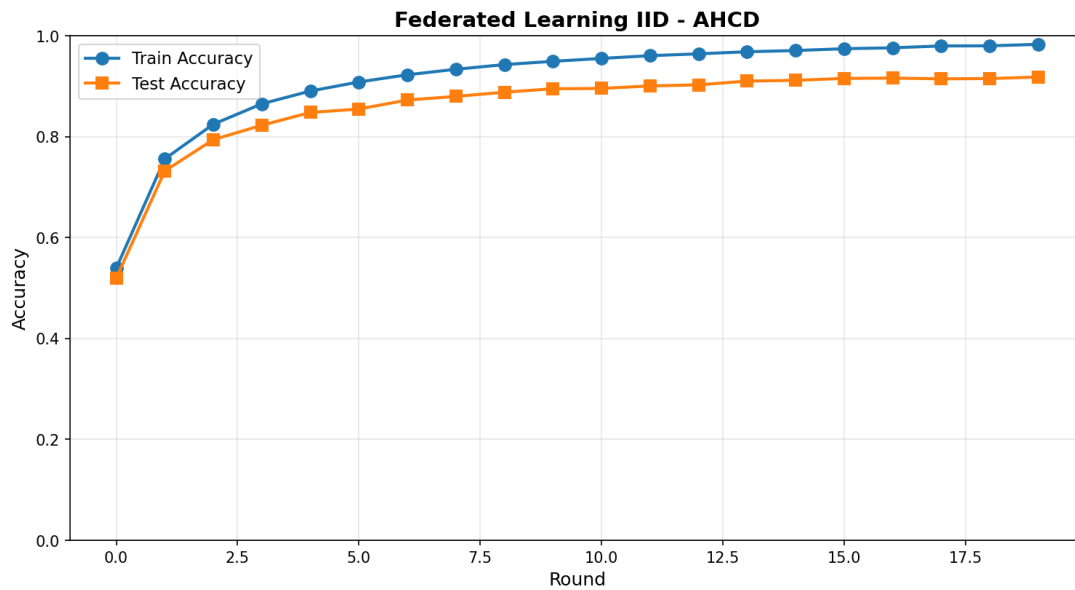


FIGURE 1 – Convergence FL IID AHCD - 20 Rounds

Les courbes montrent une convergence rapide et stable. La precision d'entraînement et de test convergent progressivement vers 91.82%, avec le plateau atteint autour du round 10.

18.2 Convergence FL Non-IID - AHCD

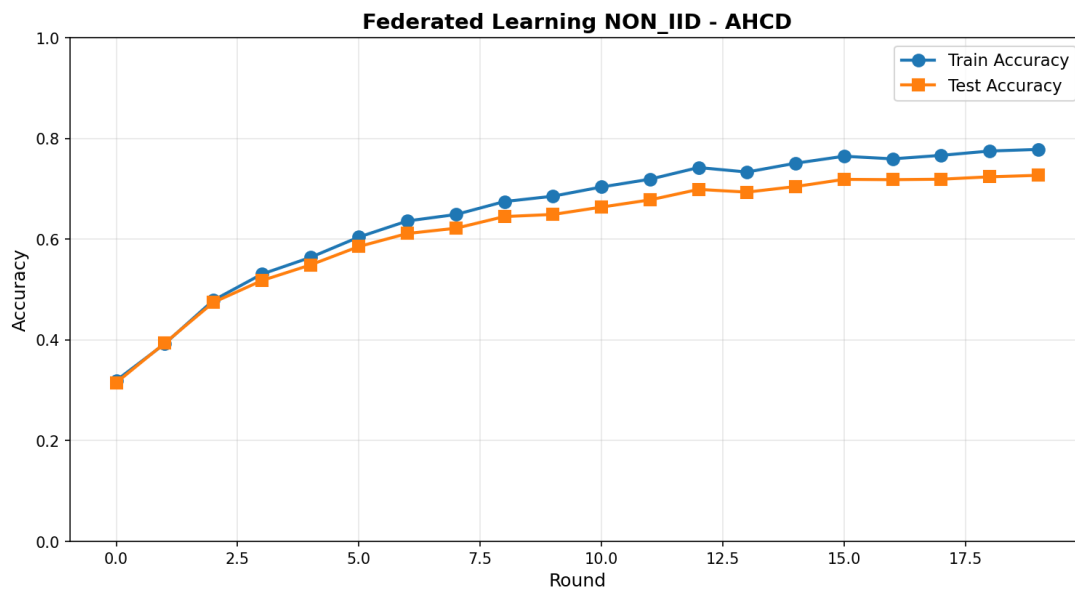


FIGURE 2 – Convergence FL Non-IID AHCD - 20 Rounds

Le scenario Non-IID montre une convergence beaucoup plus lente et un plateau bas a 72.68%, representant l'impact majeur de l'heterogeneite des donnees.

18.3 Comparaison IID vs Non-IID - AHCD

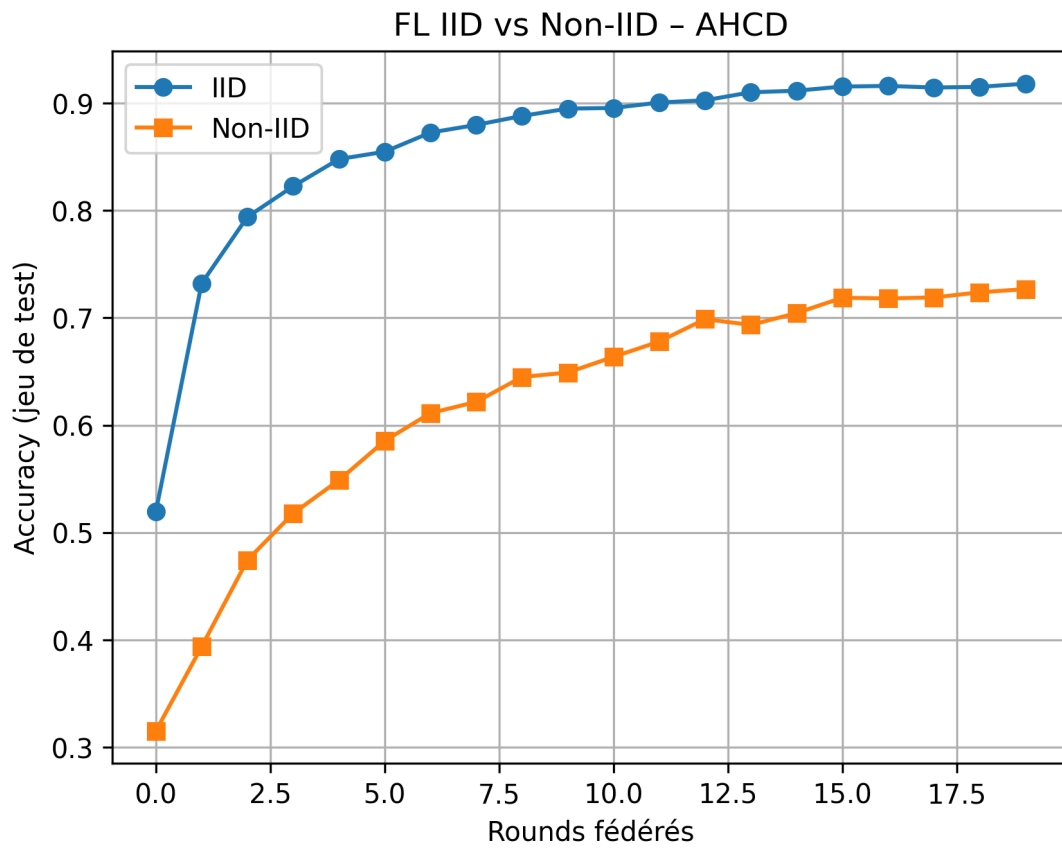


FIGURE 3 – Comparaison IID vs Non-IID - AHCD

Cette visualisation synthétise clairement l'avantage de l'approche IID sur Non-IID, mettant en évidence le challenge majeur du FL dans les scénarios realistes d'heterogeneite.

18.4 Comparaison Finale - AHCD

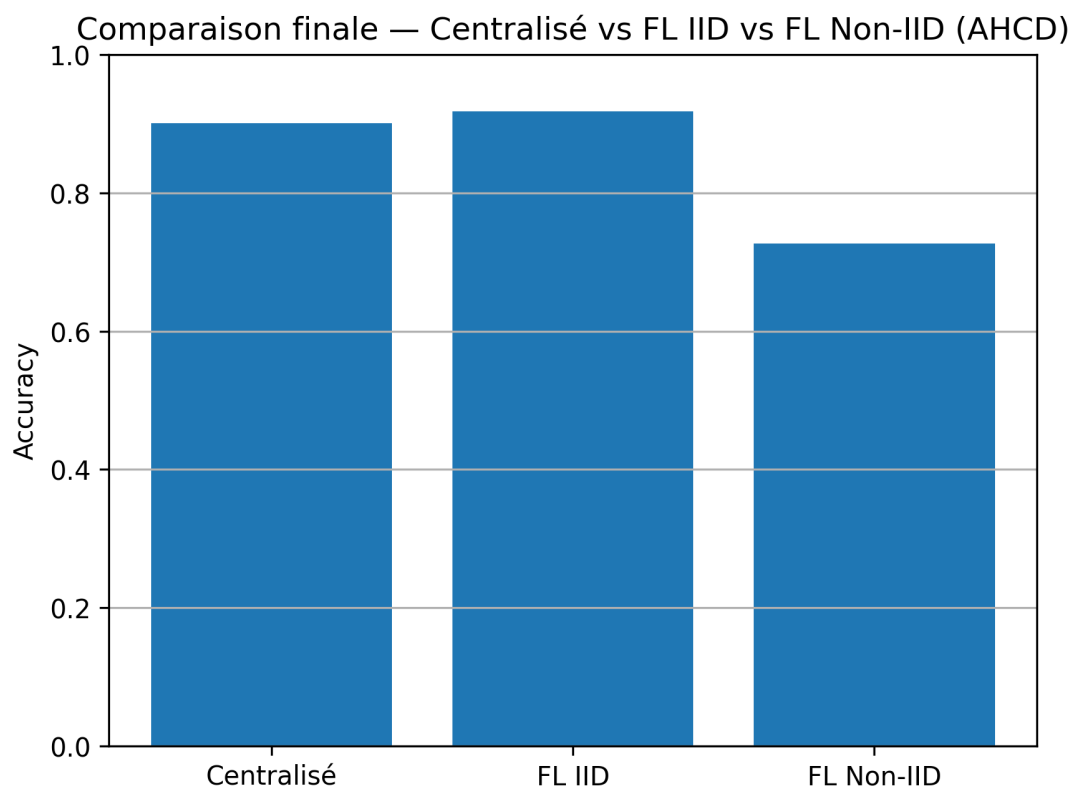


FIGURE 4 – Comparaison Finale AHCD

Le graphique compare les trois approches : Centralise (90.06%), FL IID (91.82%), et FL Non-IID (72.68%).

18.5 Comparaison OIHACDB

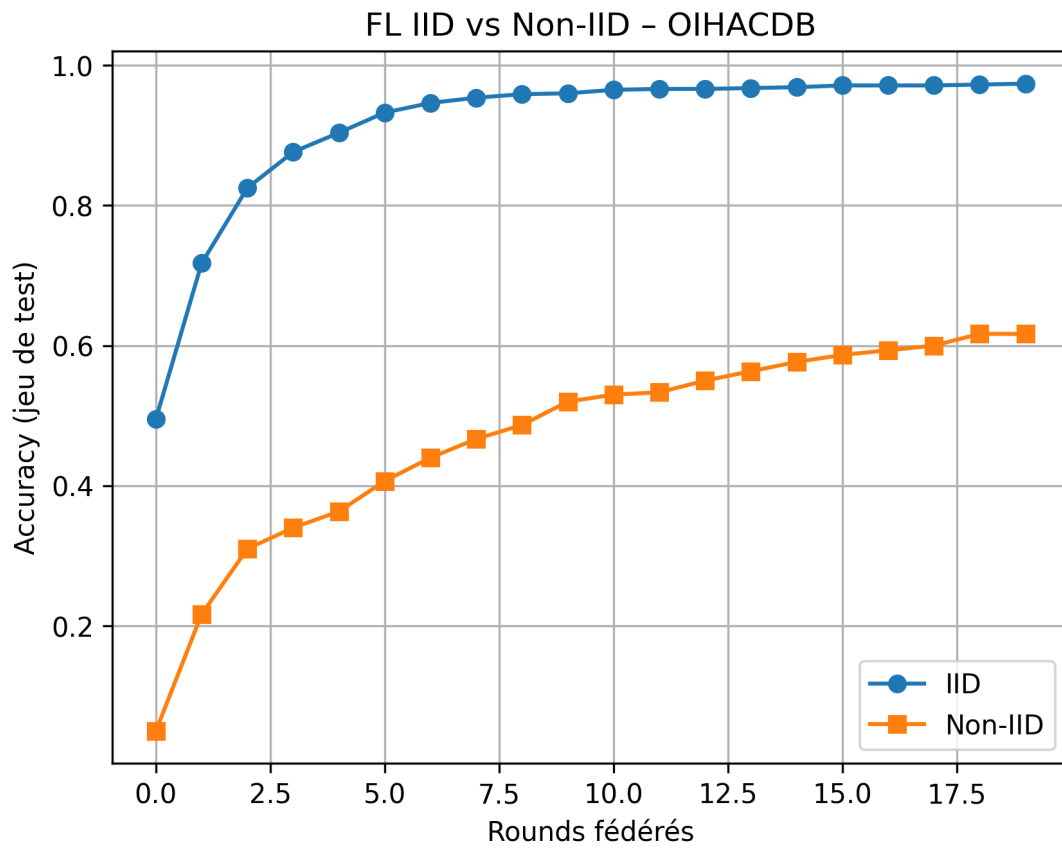


FIGURE 5 – Comparaison IID vs Non-IID - OIHACDB

OIHACDB montre une sensibilité plus importante à l'hétérogénéité, avec une dégradation de 36% (de 98% à 62%), comparée à 19.14% pour AHCD.

18.6 Master Comparison

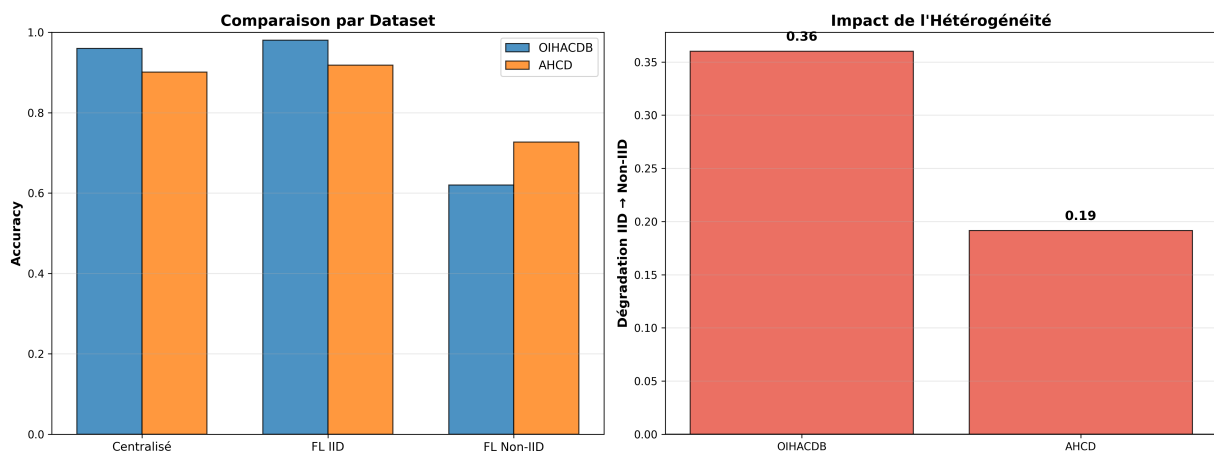


FIGURE 6 – Master Comparison - AHCD vs OIHACDB

Le graphique master compare les deux datasets et montre clairement que OIHACDB est plus sensible a l'heterogeneite que AHCD.

Conclusion

19 Resume des Resultats

19.1 Resultats Cles

- **FL IID Surpasse Centralise** : +1.76% (AHCD) et +2% (OIHACDB)
- **FL Non-IID Souffre Severement** : -19.14% (AHCD) et -36% (OIHACDB)
- **Heterogeneite = Goulot Majeur** : Reduit precision de 20-37% (relatif)

19.2 Efficacite de FedAvg

FedAvg s'est avere tres efficace pour :

- Scenarios IID : Convergence rapide et stable (20 rounds suffisant)
- Agregation distribuee : Meilleure generalisation que centralise
- Scalabilite : Fonctionne bien avec 10 clients

Cependant, FedAvg presente des limitations :

- Faible performance avec donnees Non-IID
- Necessiterait optimisations (momentum, variance reduction)
- Requier plus de rounds pour convergence en Non-IID

20 Contributions

Les contributions principales de ce travail incluent :

- Implementation complete d'un systeme FL pour la reconnaissance d'images arabes
- Evaluation systematique de scenarios IID et Non-IID
- Analyse comparative detaillee centralise vs federe
- Code open-source et reproductible

21 Limitations

- Performance degradee en Non-IID (72.68% vs 91.82% en IID)
- Scalabilite limitee a 10 clients (pas d'evaluation avec 100+ clients)
- Pas de mecanismes de confidentialite avances (DP, compression)
- Entrainement local simple (pas de momentum ou techniques avancees)

22 Travaux Futurs

- **Confidentialite Differentielle** : Ajouter du bruit gaussien pour protection renforcee
- **Compression** : Implementer compression des poids pour reduire communication
- **Selection de Clients** : Utiliser selection probabiliste
- **Agregation Robuste** : Detecter clients malveillants
- **Modeles Personalises** : Adapter modele global pour chaque client
- **mise en placetures Avancees** : Tester ResNet, MobileNet

23 Remarques Finales

L'Apprentissage Federe represente une approche prometteuse pour entrainer des modeles d'apprentissage profond sur des donnees sensibles sans les centraliser. Notre implementation de FedAvg pour la reconnaissance de caracteres arabes demonstre la viabilite pratique de cette approche tout en ouvrant des avenues interessantes pour les recherches futures en apprentissage distribue et securise.

Conclusion

Dans ce travail, nous avons étudié l'application de l'Apprentissage Fédéré à la reconnaissance des caractères manuscrits arabes isolés. Les résultats obtenus montrent que l'approche fédérée permet d'atteindre des performances comparables, voire supérieures, à l'apprentissage centralisé, tout en préservant la confidentialité des données.

Les expérimentations réalisées sur les bases AHCD et OIHACDB confirment l'impact significatif de la distribution des données, notamment dans les scénarios Non-IID où la convergence est plus difficile. Malgré ces contraintes, l'algorithme FedAvg démontre une bonne capacité d'adaptation dans un environnement distribué.

Ce travail met en évidence le potentiel de l'Apprentissage Fédéré pour les applications sensibles aux données, et ouvre la voie à de futures améliorations, telles que l'optimisation de la communication ou l'utilisation de modèles plus avancés.

Références

- [1] McMahan, B., Moore, E., Ramage, D., Hampson, S., Arcas, B. A. y. (2017). Communication-Efficient Learning of Deep Networks from Decentralized Data. In International Conference on Machine Learning (ICML).
- [2] Kairouz, P., McMahan, H. B., Aledem, B., Song, B., Manzini, K., Bonawitz, K. (2019). Advances and Open Problems in Federated Learning. arXiv preprint arXiv :1912.04977.
- [3] Zhao, Y., Li, M., Lai, L., Shalev-Shwartz, S., Song, D., Schapire, R. E. (2018). Federated Learning with Non-IID Data. arXiv preprint arXiv :1806.04522.
- [4] El-Sawy, A. E., ElBakry, H. M., Loey, M. (2016). CNN for Handwritten Arabic Digits Recognition Based on LeNet-5. In International Conference on Advances in Computer Science and Electronics Engineering (ICCSEE).
- [5] TensorFlow Team. (2024). TensorFlow : Large-Scale Machine Learning on Heterogeneous Systems. Available at : <https://www.tensorflow.org>
- [6] Chollet, F., et al. (2024). Keras : The Python Deep Learning API. Available at : <https://keras.io>